p0900r0
An Ontology for Properties of `mdspan`

Published Proposal, 12 February 2018

This version:
https://kokkos.github.io/array_ref/proposals/P0900.html

Author:
David S. Hollman

Audience:
LEWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Abstract

This paper is an initial exploration of approaches for the general specification of a properties customization point; such as proposed for mdspan [P0009r4], span [P0546r1], and other proposals. Considerations and constraints on the interaction between disparate property customizations are examined.

Table of Contents

§ 1. Background and Motivation

[P0009r4] proposes the `mdspan` class template for viewing contiguous spans of objects through a multidimensional index space. That paper also proposes that the `mdspan` class template accept `Properties...` template parameters as an extensible set of options for multi-index mapping and memory access:

```
namespace std {
namespace experimental {
  template< typename DataType , typename ... Properties >
  class mdspan ;
}}
```

[P0546r1] proposes that `span` should also accept a similar set of properties. For brevity, we will herein refer to the customization point expressed via the `Properties...` template parameter(s) of `mdspan` (and, pending [P0546r1], `span`) as *properties*. LEWG recommended that [P0019r5] should be redone as a property (on `span`, though it would probably also apply to `mdspan`), and P0860 does this. P0856 also explores the idea of such a property for expressing ISO-C `restrict`-like semantics. [P0367r0] also proposed a number of properties in a more general context, but the content is relevant to `span` and `mdspan` properties as proposed here.

We now formally explore the design of the proposed *properties* customization point. In particular, the referenced papers propose a properties extension point, define several need useful properties, and allow the application of multiple properties to the same type. Thus the design for how multiple properties may interact must be defined; for example,

- What sets of properties are allowed to be given together?
    - Does the addition of a property require the developer to define the set of properties with which it is compatible? How do we prevent exponential explosion of work from this requirement?
- What does it *mean* when multiple properties are given together?
    - Again, does the addition of a property require the developer to define the set of behaviors when combined with all other sets of properties? How do we prevent exponential explosion of work from this requirement?
- Are the behaviors of a `span` or `mdspan` dependent on the order of the properties given? If so, what is the meaning of the order?

We try to address these questions and other related issues by sketching an ontology for `mdspan` (and `span`) properties herein.

§ 2. Design Principles and Decisions

What follows is a set of overarching principles for the design of the property customization points. These principles suggest a set of restrictions to the set of features implementable via the customization point, and an exploration of the viability of these restrictions follows, particularly in the context of the properties proposed (or suggested) thus far in other papers.

§ 2.1. Orthogonality

The issue of exponential explosion for property compatibility specification work is so dire (particularly with respect to vendor-supplied or user-supplied extensions) that we suggest taking extreme measures. Thus, we propose that **all** properties should be allowed with all other properties, and that any property that cannot tolerate this level of compatibility should *not* use this customization point but should be implemented via some other mechanism. The implications of this constraint are explored below, and a mechanism for implementing and enforcing this constraint is also proposed herein. Some potential for nuance within this restriction is also discussed below.

A corrollary to this principle is that the behavior and attributes of an `mdspan` under a set of properties should be expressible as a well-defined combination of the behaviors or attributes of the `mdspan` under each of those properties individually. For instance, the `noexcept` specifier for `mdspan<T, Properties...>::operator[](index_type)` should be expressible as:

```
noexcept(conjunction_v<
  boolean_constant<noexcept(declval<mdspan<T, Properties>>()[std::declval<i
>)
```

Similar combination should be possible for behaviors via the specification of hooks in a traits-like interface, as explored below.

§ 2.2. Property Modes

It is important to distinguish between property "types" and their "values". For instance, [P0367r0] proposes a rich set of read/write access qualifiers, including the expected `read`, `write`, and `read_write`, but also including more specialized "values" like `discard_read` and `discard_write`. It is clear that these "values" are meant to be mutually exclusive; that is, it would be nonsensical to have an `mdspan` with both `write` and `read_write`. If a flat model were used for the property customization point, this nonsensical mutual exclusivity would violate the orthogonality design principle above. However, if we instead view these as different "modes" (or "values") of a single "property", orthogonality can be preserved while still providing the mutual exclusivity needed to properly express this property. In this example, `read`, `write`, etc., would be "values" of a property named something like `access_mode`. To avoid confusion with the loaded terms "type" and "value" that have other meanings in C++, we will use the term "property" to refer to the group of mutually exclusive property "values", and "property mode" to refer to the "value" of that property. Note that most properties (at least of those proposed thus far) will have a mode type that is merely a boolean, with their mode indicated by their presence or abscence in the `Properties...` parameter pack. Another example of a non-boolean modal property is the layout property proposed in [P0009r4].

§ 2.3. Convertibility

A follow-on to the orthogonality design decision is that of convertibility (via construction or assignment). In similar interest of avoiding exponential explosion of pairwise convertibility specification and implementation, it makes sense to impose the constraint that an `mdspan` with *any* set of properties is always convertible to an `mdspan` with any other set of properties (and likewise for

`span`). A slight nuance that could be added to this constraint is to allow properties to forbid interconversion between different *modes* within the same property. For instance, with the access mode property proposed in [P0367r0], if the `read` mode of that property is specified to mean that only reads will be done from the data in the `mdspan` or objects derived from it, then it is a clear violation of the contract to convert such an `mdspan` into one with the `write` mode of that property. It is not unreasonable to allow this prohibition to be made within the type system, since the specification complexity is always linear in the number of modes of the property, and since the modes of a given property are not subject to the same extensibility constraints as properties themselves.

§ 2.4. On the Importance of API Invariance

In the design of many customization points analogous to the properties proposed herein, invariance of the API for members of the resulting class template instantiation is critical to its usefulness in a generic context. For example, the invariance of the `vector` API with respect to the `Allocator` template parameter customization point makes the following generic code viable:

```
template <typename T, typename Allocator>
auto copy_odd_values(const vector<T, Allocator>& v) {
    vector<T, Allocator> ret;
    auto is_odd = [](T const& i) { return i % 2 != 0; };
    std::copy_if(v.begin(), v.end(), std::back_insert_iterator(ret), is_odc
    return ret;
}
```

In the context of properties for `span` and `mdspan`, this invariance is actually not quite as critical for several important reasons. Most importantly, both `span` and `mdspan` are intended to be passed by value. Thus, there is less need to template on the `Properties...` template parameters, for example in order to avoid spurious copies to `const` reference parameters. In fact, some properties represent a contract that would be dangerous to transmit into a generic context, as is the case with `restrict_access` (proposed in P0856). In such cases, it actually makes more sense to eschew support for a generic properties variadic template parameter in favor of specific enumeration of the properties known to be supported by the generic function. Many compiler-specific properties are likely to also represent similar sorts of contracts.

§ 2.5. On Transmission into Generic Contexts

Unfortunately, for other proposed properties, transmission into and through a generic context *is* desirable. The `bounds_checking` and `layout` properties suggested in [P0009r4] should be transmitted without issue into most generic contexts. Without loss of extensibility (i.e., without doing things like explicitly listing the properties that may be transmitted, which precludes the use of vendor-specific or user-specific properties), it is not easy to write code that is generic with respect to some properties but not others. A potential solution for this is explored below.

§ 3. Proposed Solution: Property Traits

One way to implement many of the design constraints expressed above is to specify a mechanism for the application of properties to a `span` or `mdspan` that maintains adherence to these design principles through its action. As such, we propose a set of property traits via which most of the aspects of most of the properties proposed thus far can be expressed. (Those that express contracts or hints to compilers and optimizers obviously cannot be fully specified in this way, but even in those cases some of these traits may be relevant.) The traits in this section are expressed at namespace scope, in keeping with newer interface proposals such as the one in [P0443R3], rather than at class scope (as is the case with `allocator_traits`), but they could easily be refactored to a different form based on feedback. The set of traits proposed here is not intended to be exhaustive; rather it is intended to be sufficient to implement the properties previously proposed or suggested and most properties foreseeable based on that experience.

§ 3.1. Traits for Member Types

There are two possible approaches to resolving conflicts between properties attempting to modify the same member type. The first approach is to apply the type trait to each property in a nested manner, using the order the properties are given in the `Properties...` list (though the best practice should be that the result of this operation should be invariant with respect to order). For example, `mdspan<T, Properties...>::element_type` would be defined in terms of a recursive application of the following customization point:

```
namespace std {
namespace experimental {
namespace properties {

template <class SpanLike, class Property>
struct element_type;
template <class SpanLike, class Property>
using element_type_t = typename element_type<SpanLike, Property>::type;

}}}
```

where `SpanLike` is a `span` or `mdspan` with all properties preceding `Property` in the `Properties...` list. For clarity, here is a possible implementation of the `element_type` member type of `mdspan`:

```
namespace std {
namespace experimental {

template <class DataType, class... Properties>
class mdspan;

template <class DataType, class Property, class... Properties>
class mdspan {
public:
  using element_type =
    properties::element_type_t<mdspan<DataType, Properties...>, Property>>;
};

// Base case is the mdspan without any properties:
template <class DataType>
class mdspan {
public:
  using element_type = remove_all_extents_t<DataType>;  // as in P0009r4
  /* ... */
};

}}
```

and the default, unspecialized implementation of the customization point for `element_type` would be:

```
namespace std {
namespace experimental {
namespace properties {

template <class SpanLike, class Property>
struct element_type { using type = typename SpanLike::element_type; };

}}}
```

The rest of the element types would have similar customization points:

```
namespace std {
namespace experimental {
namespace properties {

template <class SpanLike, class Property>
struct value_type;
template <class SpanLike, class Property>
using value_type_t = class value_type<SpanLike, Property>::type;

template <class SpanLike, class Property>
struct index_type;
template <class SpanLike, class Property>
using index_type_t = class index_type<SpanLike, Property>::type;

template <class SpanLike, class Property>
struct difference_type;
template <class SpanLike, class Property>
using difference_type_t = class difference_type<SpanLike, Property>::type;

template <class SpanLike, class Property>
struct pointer;
template <class SpanLike, class Property>
using pointer_t = class pointer<SpanLike, Property>::type;

template <class SpanLike, class Property>
struct reference;
template <class SpanLike, class Property>
using reference_t = class reference<SpanLike, Property>::type;

}}}
```

3.1.1. On order dependence and order invariance

This approach has the disadvantage of imposing a theoretical order dependence on the properties customization point. Practically speaking, it would be considered a best practice of a high-quality implementation to ensure that a property's implementation of `properties::element_type_t` (and other traits) are invariant with respect to ordering with other known properties (such as those in the standard library), but with this approach it would be impossible to make that guarantee formal, particularly with respect to other vendor-defined and user-defined properties unknown to the property

implementer. This "best practice" enforcement of order invariance is, in some sense, also an advantage of this specifying the customization point action in this way, since in the event of an unavoidable ordering dependency the behavior is both well-defined and easily explained in terms of a relatively simple mechanism. This also makes sense in terms of defining an order for invocation of hooks (discussed below), which may be needed to implement some properties.

3.1.2. Alternative means of resolving conflicts

A second approach to resolving conflicts between two properties customizing the same member type trait (or most of the other traits) is to disallow it and to make this the *definition* of property incompatibility. Other than eliminating order dependence, this approach has the advantage of producing a clean, easy-to-explain definition of property incompatibility. However, there are several key disadvantages to this approach. First, the implementations of many property traits will be implementation-defined, which implies that one implementation may specialize a trait for a given property and another may use a different mechanism (e.g., specialization of a different trait). This leads to the uncomfortable scenario where either the mechanism for implementation of any standardized properties needs to be specified, or (much worse) the compatibility between properties is implementation-defined. Beyond this, not all mututally exclusive modes of a given property may modify the same property traits. For instance, in the example above with `read`/`write`/`read_write`/`read_discard`/etc. modes of some property (which we called `access_mode` for the sake of discussion), the `read` and `read_discard` modes would likely specialize `properties::reference_t` to be a `const` reference to the `value_type` of the `span` or `mdspan`, but there is no equivalent specialization for the `write` mode. Thus, to enforce the mutual exclusivity that is obvious from the property's definition, one would need to define a special case that specifies incompatibility *in spite of* trait-wise compatibility.

3.2. Data Access Hooks

Similar to the property member type traits from above, it is desirable to define the effects of properties on `span` and `mdspan` methods in a manner that is both orthogonal to that of other properties and separate from the implementation of the property-free implementations for `span` and `mdspan`. Applying the same namespace-scope traits pattern in, e.g., [P0443R3], we can define ADL-accessible hooks that property implementers would customize to implement the property's behavior.

```
namespace std {
namespace experimental {
namespace properties {

template <class SpanLike, class Property>
typename SpanLike::index_type_t
pre_subscript_operator_hook(SpanLike s, Property p, class SpanLike::index_t
template <class SpanLike, class Property>
void post_subscript_operator_hook(SpanLike s, Property p, class SpanLike::i

template <class SpanLike, class Property, class... IndexType>
std::tuple<IndexType&&...>
pre_call_operator_hook(SpanLike s, Property p, IndexType&&... idxs) noexcep
template <class SpanLike, class Property, class... IndexType>
void post_call_operator_hook(SpanLike s, Property p, IndexType&&... idxs) r

template <class SpanLike, class Property, class IndexType, size_t N>
array<IndexType, N> const&
pre_call_operator_hook(SpanLike s, Property p, array<IndexType, N> const& i
template <class SpanLike, class Property, class IndexType>
void post_call_operator_hook(SpanLike s, Property p, array<IndexType, N> cc

}}}
```

The intent is that the pre-invocation hooks would take the arguments of the method invocation and return the (potentially modified) arguments to be used with the next property hook or the implementation of the method in the property-free `span`/`mdspan` implementation of that method. For instance, a possible implementation of `mdspan<T, Properties...>::operator()` would be

```
template <class DataType, class Property, class... Properties>
class mdspan {
public:
  template <class IndexType, size_t N>
  reference operator()(array<IndexType,N> const& indices) const noexcept {
    // remove Property and call the next less-property-qualified implementa
    // Optimizer should be able to remove spurious copy here (if not, inher
    // could be used internally to achieve the same effect)
    auto rv = mdspan<DataType, Properties...>(*this).operator()(
      // intentionally use the unqualified type to trigger ADL
      pre_call_operator_hook(*this, Property{}, indices)
    );
    post_call_operator_hook(*this, Property{}, indices);
    return rv;
  }
  template <class... IndexType>
  reference operator()(IndexType&&... indices) const noexcept {
    // similar, but slightly more complicated because of the variadic templ
    auto rv = std::apply(
      [this](IndexType&&... idxs){
          mdspan<DataType, Properties...>(*this).operator()(std::forward<Ir
      },
      pre_call_operator_hook(*this, Property{}, std::forward<IndexType>(inc
    );
    post_call_operator_hook(*this, Property{}, std::forward<IndexType>(indi
    return rv;
  }
};
```

Again, the question of order-invariance arises, though in this case it is much harder to detect the presence or absense of a customization for a given property (if an order-independent approach is favored in spite of the issues raised in §3.1.2 Alternative means of resolving conflicts, it may be desirable to use the "monolithic traits class" approach instead, similar to the approach of `allocator_traits`). Though best practice should be that the actions taken by these hooks are order independent, guaranteeing this for any one property may not be reasonable. Additionally, even if the actions of two hooks are entirely independent and could be reordered by the compiler, it may be useful to understand the hook application order for performance purposes.

§ 3.3. Layout-Related Traits and Hooks for `mdspan` Only

```
namespace std {
namespace experimental {
namespace properties {

template <class MDSpanLike, class Property>
struct is_always_unique;
template <class MDSpanLike, class Property>
inline constexpr bool is_always_unique_v = is_always_unique<MDSpanLike, Pro
template <class MDSpanLike, class Property>
constexpr bool is_unique(MDSpanLike s, Property p);


template <class MDSpanLike, class Property>
struct is_always_contiguous;
template <class MDSpanLike, class Property>
inline constexpr bool is_always_contiguous_v = is_always_contiguous<MDSpanL
template <class MDSpanLike, class Property>
constexpr bool is_contiguous(MDSpanLike s, Property p);

template <class MDSpanLike, class Property>
struct is_always_strided;
template <class MDSpanLike, class Property>
inline constexpr bool is_always_strided_v = is_always_strided<MDSpanLike, P
template <class MDSpanLike, class Property>
constexpr bool is_strided(MDSpanLike s, Property p);

template <typename MDSpanLike, class Property>
constexpr index_type_t<MDSpanLike> stride(MDSpanLike s, Property p, int i);

}}}
```

The layout-related property traits needed to implement `mdspan` are a bit of a special case. It doesn't really make sense to talk about what happens when multiple properties specialize these traits, and trying to formulate these traits in a form that can be applied multiple times leads to more confusing and less intuitive definitions of the traits (e.g., one could imagine that `mdspan<T, Properties...>::stride()` is implemented in terms of the sum of the return values from the invocations of the customization points with each of the properties separately, and the default implementation of the customization point for properties that don't care about stride could return 0).

There are a few approaches to explore for addressing this inconsistency. Taking the approach that no two properties given for the same `span`/`mdspan` are allowed to specialize the same customization point makes layout-like properties more regular, and perhaps this increase in simplicity is enough to outweigh the disadvantages discussed in §3.1.2 Alternative means of resolving conflicts. Another option is to provide some sort of trait that says whether or not a property expects to be the only one specializing its customization points; something like `is_unique_customization_v`. This is the most complex solution, but provides the most flexibility for future property extensions. It also lends itself to an progressive specification strategy by which new properties "lock down" there customization point specializations until a compelling use case is presented requiring interoperability. (A more aggressive version of this approach may even specify the expected uniqueness of the customization on a per-trait basis.) A third option is to just exclude layout-like properties from the ontology entirely and instead treat it as part of the base `mdspan` class template. This third approach seems sensible in light of the fact that layout properties don't really apply to `span`, so its inclusion in the wider ontology may be unnecessary. This third strategy almost certainly makes the most sense for the `extents` property proposed in [P0009r4], since it is mandatory for `mdspan`, affects most of the `mdspan` implementation, and does not apply to `span`.

§ 3.4. Traits for Propagation into or through Generic Contexts

As discussed above in §2.5 On Transmission into Generic Contexts, it is desirable for generic function templates to deduce and propagate some properties, while for others it is dangerous to do so. One potential solution to this problem that doesn't require an "all-or-nothing" approach is to have individual properties specify whether or not they should be propagated through a generic context that otherwise makes no mention of the property. For instance, a generic function template that takes two `mdspan` arguments should never deduce `restrict_access` as a property of its arguments unless that function is explicitly implemented to obey the `restrict_access` contract. For instance, consider a generic, non-`restrict_access`-aware function template that calls a generic, `restrict_access`-aware dot product function (implemented with separate overloads for `restrict_access` arguments and non-`restrict_access` arguments):

```
template <class T1, class... Properties1, class T2, class... Properties2>
auto my_generic_function(mdspan<T1, Properties1...> s1, mdspan<T2, Properti
    return generic_dot(s1, s1) + generic_dot(s2, s2);
}
```

If `restrict_access` is deduced to be one of the properties of `s1` or `s2`, then `my_generic_function` has unknowingly called the incorrect overload of `generic_dot`, causing it

to potentially violate the contract implied by the `restrict_access` property. If, on the other hand, the arguments `s1` and `s2` have the `atomic_access` property enabled (see P0860), it is clearly the intent of the caller that `my_generic_function` and all of its callees make use of the underlying data via atomic loads, stores, etc. (likely implemented via a wrapper to the `reference_t`; see details in P0860). The generic context should thus preserve the `atomic_access` property but discard the `restrict_access` property.

One potential solution to this problem is to introduce a trait that returns the mode of the property intended for propagation through a generic interface. The trait would look something like:

```
namespace std {
namespace experimental {
namespace properties {

template <class Property>
using generic_context_propagation_mode_t = /* implementation defined */;

}}}
```

Implementers could then rewrite `my_generic_function` as:

```
template <class T1, class... Properties1, class T2, class... Properties2>
auto my_generic_function(mdspan<T1, Properties1...> s1, mdspan<T2, Properti
    auto s1g = mdspan<T1, properties::generic_context_propagation_mode_t<Pr
    auto s2g = mdspan<T2, properties::generic_context_propagation_mode_t<Pr
    return generic_dot(s1g, s1g) + generic_dot(s2g, s2g);
}
```

Supposing that `restrict_access` is short for something like `restrict_access_enabled<true>`, the trait `generic_context_propagation_mode_t` could be specialized for `restrict_access` to return `restrict_access_enabled<false>`.

Another option is to discourage deduction of property template parameters and rely on conversion and compatibility to prevent accidental failure to propagate a property. For instance, the `atomic_access` property could prohibit casting away `atomic_access` to avoid accidental usage via non-atomic operations. This means that library implementers would have to implement more overloads that they otherwise would, but the simplification may be worth it. The need for implementation of multiple overloads when different properties are present may exist anyway, since `atomic_access` changes the type (and thus the API) of the reference returned by the subscript and call operators. Though this

option somewhat limits vendor-specific extensibility (and particularly limits user-specific extensibility), it appears to be reasonable in the contexts of the properties proposed thus far.

§ 3.5. Traits for Expression of Convertibility

While it seems wise to require all `mdspan` instantiations be convertible to and from all `mdspan` instantiations with different properties, it is reasonable for properties to restrict which of their modes are interconvertible. This, then, requires trait customization points for expressing whether two property modes belong to the same property as well as whether copy operations are allowed between any pair of modes of the same property.

```
namespace std {
namespace experimental {
namespace properties {

template <class PropertyInMode1, class PropertyInMode2>
inline constexpr bool modes_of_same_property_v = /* implementation defined

template <class PropertyInMode1>
using default_mode_for_property_t = /* implementation defined */

template <class PropertyInMode1, class PropertyInMode2>
inline constexpr bool are_convertible_property_modes_v = /* implementation

}}}
```

With this definition, there is some question as to whether the property itself should be required to have some sort of identifying tag that, for instance, can be used with `is_same` to determine if two property modes are of the same property. This topic can be discussed further as the abstractions proposed herein are hardened.

§ 3.6. Other Potential Traits and Hooks

Other places where traits or hooks may need to be placed in the future include post-construction, pre- and post-assignment, pre- and post-iterator creation (`span` only), iterator member type (`span` only), comparison (`span` only), subspan creation, and `span` conversion (`mdspan` only). Traits and hooks for these customization points have not been included here because there has not yet been a property

proposed that would make use of these. The patterns established herein for the other traits and hooks should be reasonably applicable to these cases, however.

§ 4. Open Questions and Straw Polls

- Should `extents` (and to a lesser extent, `layout`) from [P0009r4] be made to fit in this property ontology, or should these be part of the `mdspan` class template itself (and thus subject to different rules). (Stated differently, should the property ontology and traits proposed herein be adapted to accomodate `extents` and `layout`?)
- Straw poll: Should the customization of the same trait/hook by multiple properties be allowed (with conflicts resolved via order dependence), or should customization point conflicts be used as an indicator of incompatibility?
- Straw poll: If customization of the same trait/hook by multiple properties is allowed, should a trait like `is_unique_customization_v` be provided that prohibits multiple customization on a case-by-case basis?
- Straw poll: Should the traits customization points proposed here be implemented at namespace scope or as part of a monolithic traits class?
- Straw poll: Should the guidance be to propagate properties via the instructions of a trait like `generic_context_propagation_mode_t`, or should the guidance be to not propagate properties via template argument deduction?

Active content removed

§ References

§ Informative References

[P0009r4]
H. Carter Edwards, Bryce Lelbach, Christian Trott, Mauro Bianco, Robin Maffeo, Ben Sander, Athanasios Iliopoulos, John Michopoulos. Polymorphic Multidimensional Array Reference. URL: https://wg21.link/p0009r4

[P0019r5]
H. Carter Edwards, Hans Boehm, Olivier Giroux, James Reus. Atomic View. URL: https://wg21.link/p0019r5

[P0367r0]
Ronan Keryell, Joël Falcou. a C++ standard library class to qualify data accesses. 29 May 2016. URL: https://wg21.link/p0367r0

[P0443R3]

Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, Carter Edwards, Gordon Brown. A Unified Executors Proposal for C++. URL: https://wg21.link/p0443r3

[P0546r1]

Carter Edwards, Bryce Lelbach. Span - foundation for the future. URL: https://wg21.link/p0546r1